

ECO-WATT API Documentation - Team Saturated

Overview

The ECO-WATT API provides endpoints for solar inverter data upload, real-time monitoring, and system statistics. The API handles compressed solar data with 9:1 compression efficiency and publishes real-time data via MQTT, and Node-Red froat end is used to dislay the real time results.

Base Service URL

```
Flask: http://localhost:5000
Node-Red : http://localhost:1880/ui/
```

Authentication

Currently using open access for development. Production deployment should implement API key authentication.

Upload API

POST /api/inverter/upload

Upload compressed solar inverter data from ESP32 devices.

Content-Type: application/json or application/octet-stream

Request Format (JSON)

```
{
  "data": "0F0A00E85604E85618E8562CE85640E85654E85668E8567CE85690E856A4E856B8E856CCE856E0E856F4E85708",
  "device_id": "ESP32_SOLAR_INV_001",
  "size": 314
}
```

Request Format (Binary)

Raw binary compressed payload (314 bytes)

Request Parameters

Parameter	Type	Required	Description
data	string	Yes (JSON)	Hex-encoded compressed solar data
device_id	string	Yes	Unique ESP32 device identifier
size	integer	Yes	Size of compressed data in bytes

Response (Success)

```
{
  "status": "success",
  "message": "Data uploaded and published to MQTT",
  "compression_stats": {
    "original_bytes": 2880,
    "compressed_bytes": 314,
    "ratio": 9.17,
    "space_saved_percent": 89.1
  },
  "records_processed": 15,
  "mqtt_published": true,
  "processing_time_ms": 45,
  "timestamp": "2025-09-28T14:30:45.123456Z"
}
```

Response (Error)

```
{
  "status": "error",
  "message": "Invalid data format",
  "error_code": "DECOMPRESSION_FAILED",
  "timestamp": "2025-09-28T14:30:45.123456Z"
}
```

Status Codes

- 200 OK - Upload successful
- 400 Bad Request - Invalid data format or missing parameters
- 413 Payload Too Large - Data exceeds size limits
- 500 Internal Server Error - Server processing error

Dashboard API

GET /api/dashboard/health

Get server health status and connection information.

Response

```
{
  "server_status": "running",
  "mqtt_connected": true,
  "uptime_seconds": 3600,
  "last_upload": "2025-09-28T14:30:45Z",
  "total_uploads": 240,
  "error_count": 0,
  "memory_usage_mb": 45.2,
  "version": "1.0.0"
}
```

GET /api/dashboard/stats

Get comprehensive upload and compression statistics.

Response

```
{
  "current_session": {
    "uploads_count": 42,
    "average_compression_ratio": 9.15,
    "total_data_saved_mb": 15.3,
    "last_device_id": "ESP32_SOLAR_INV_001",
    "session_start": "2025-09-28T10:00:00Z"
  },
  "compression_performance": {
    "best_ratio": 9.42,
    "worst_ratio": 8.86,
    "average_processing_time_ms": 38,
    "total_bytes_processed": 132480
  },
  "mqtt_stats": {
    "messages_published": 42,
    "publish_success_rate": 100.0,
    "last_publish": "2025-09-28T14:30:45Z",
    "broker_status": "connected"
  }
}
```

GET /api/dashboard/solar-data

Get latest solar inverter readings and historical summary.

Query Parameters

Parameter	Type	Default	Description
limit	integer	1	Number of recent readings to return
device_id	string	all	Filter by specific device ID

Response

```
{
  "latest_reading": {
    "timestamp": "2025-09-28T14:30:45Z",
    "device_id": "ESP32_SOLAR_INV_001",
    "ac_voltage": 230.5,
    "ac_current": 4.2,
    "ac_power": 968.1,
    "ac_frequency": 50.0,
    "dc_voltage": 350.8,
    "dc_current": 2.8,
    "dc_power": 982.2,
    "temperature": 42.5,
    "status": 1,
    "efficiency": 98.6
  },
  "historical_summary": {
    "readings_count": 360,
    "time_range": "24 hours",
    "max_power": 1250.0,
    "min_power": 850.0,
    "average_voltage": 230.2,
    "average_temperature": 41.2,
    "peak_efficiency": 99.2,
    "total_energy_kwh": 23.4
  }
}
```

System API

GET /api/system/info

Get system information and configuration.

Response

```
{
  "system": {
    "version": "1.0.0",
    "build": "2025-09-28",
    "environment": "development"
  },
  "configuration": {
    "mqtt_broker": "broker.emqx.io",
    "mqtt_topic": "vdl/replace",
    "compression_enabled": true,
    "max_upload_size": 1024
  },
  "capabilities": [
    "compression_9_1",
    "mqtt_publishing",
    "real_time_dashboard",
    "multi_device_support"
  ]
}
```

GET /api/system/metrics

Get detailed system performance metrics.

Response

```
{
  "performance": {
    "cpu_usage_percent": 15.2,
    "memory_usage_mb": 45.8,
    "disk_usage_percent": 12.3,
    "network_io_kb": 1250
  },
  "api_metrics": {
    "total_requests": 1543,
    "average_response_time_ms": 125,
    "requests_per_minute": 4.2,
    "error_rate_percent": 0.1
  },
  "compression_metrics": {
    "total_compressions": 240,
    "average_ratio": 9.15,
    "total_space_saved_mb": 67.3,
    "compression_time_avg_ms": 12
  }
}
```

Data Formats

Solar Inverter Registers

The system processes 10 standard solar inverter registers:

Register	Name	Unit	Scale	Description
0	AC Voltage	V	÷10	AC output voltage
1	AC Current	A	÷100	AC output current
2	AC Power	W	÷10	AC output power
3	AC Frequency	Hz	÷10	AC frequency
4	DC Voltage	V	÷10	DC input voltage
5	DC Current	A	÷100	DC input current
6	DC Power	W	÷10	DC input power
7	Temperature	°C	÷10	Inverter temperature
8	Status	-	raw	System status code
9	Efficiency	%	÷10	Conversion efficiency

Compression Format

Binary data structure for compressed uploads:

```
Header: [num_records][regs_per_record]
Record: [delta_timestamp][reg0_hi][reg0_lo]...[reg9_hi][reg9_lo]

Typical payload:
- Header: 2 bytes
- Records: 15 × 21 bytes = 315 bytes
- Total: 314 bytes (compressed from 2,880 bytes)
```

MQTT Message Format

Published to topic `vd1/replace` :

```
{
  "timestamp": "2025-09-28T14:30:45.123456Z",
  "device_id": "ESP32_SOLAR_INV_001",
  "data": [
    {
      "timestamp": 1727522445,
      "ac_voltage": 230.5,
      "ac_current": 4.2,
      "ac_power": 968.1,
      "ac_frequency": 50.0,
      "dc_voltage": 350.8,
      "dc_current": 2.8,
      "dc_power": 982.2,
      "temperature": 42.5,
      "status": 1,
      "efficiency": 98.6
    }
  ],
  "compression_stats": {
    "original_bytes": 2880,
    "compressed_bytes": 314,
    "ratio": 9.17,
    "space_saved_percent": 89.1
  },
  "metadata": {
    "upload_interval": "15 seconds",
    "records_per_upload": 15,
    "total_registers": 10,
    "algorithm_version": "delta_v2"
  }
}
```

Error Codes

Upload Errors

Code	Description	Solution
INVALID_FORMAT	Data format is invalid	Check JSON structure or binary format
DECOMPRESSION_FAILED	Cannot decompress data	Verify compression algorithm compatibility
MISSING_DEVICE_ID	Device ID not provided	Include device_id in request
PAYLOAD_TOO_LARGE	Data exceeds size limit	Check compressed data size
MQTT_PUBLISH_FAILED	Cannot publish to MQTT	Check MQTT broker connection

System Errors

Code	Description	Solution
SERVER_OVERLOAD	Server under high load	Retry after delay
MQTT_DISCONNECTED	MQTT broker disconnected	Check network connectivity
STORAGE_FULL	Storage space exhausted	Clear old data or increase storage
CONFIG_ERROR	Configuration issue	Check server configuration

Rate Limiting

Current Limits

- **Upload endpoint:** 60 requests per minute per device
- **Dashboard endpoints:** 120 requests per minute
- **System endpoints:** 30 requests per minute

Rate Limit Headers

```
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 45
X-RateLimit-Reset: 1727522500
```

Examples

Upload Example (cURL)

```
# JSON upload
curl -X POST http://localhost:5000/api/inverter/upload \
-H "Content-Type: application/json" \
-d '{
  "data": "0F0A00E85604E85618E856...",
  "device_id": "ESP32_001",
  "size": 314
}'

# Binary upload
curl -X POST http://localhost:5000/api/inverter/upload \
-H "Content-Type: application/octet-stream" \
--data-binary @compressed_data.bin
```

Dashboard Example (JavaScript)

```
// Get latest solar data
fetch('/api/dashboard/solar-data')
  .then(response => response.json())
  .then(data => {
    console.log('Latest power:', data.latest_reading.ac_power);
    console.log('Efficiency:', data.latest_reading.efficiency);
  });

// Get system health
fetch('/api/dashboard/health')
  .then(response => response.json())
  .then(health => {
    console.log('Server status:', health.server_status);
    console.log('MQTT connected:', health.mqtt_connected);
  });
```

Python Client Example

```
import requests
import json

# Upload compressed data
upload_data = {
    "data": "0F0A00E85604E85618E856...",
    "device_id": "ESP32_001",
    "size": 314
}

response = requests.post(
    'http://localhost:5000/api/inverter/upload',
    json=upload_data
)

if response.status_code == 200:
    result = response.json()
    print(f"Compression ratio: {result['compression_stats']['ratio']}:1")
else:
    print(f"Upload failed: {response.json()['message']}")
```

Testing

Expected Response Times

- Upload endpoint: < 500ms
- Dashboard endpoints: < 100ms
- System endpoints: < 50ms

Changelog

Version 1.0.0 (2025-09-28)

- Initial API release
- Upload endpoint with 9:1 compression
- Dashboard endpoints for monitoring
- MQTT integration
- System health and metrics endpoints
- Comprehensive error handling

Support

Documentation

- **Server Guide:** `Server/server-guide.md`
- **API Tests:** `test/` directory
- **MQTT Testing Tools:** `mqtt_listener.py`, `mqtt_publisher.py`

Contact

- **Repository:** <https://github.com/Team-Saturated/Eco-watt>